Density estimation II

Kernel density estimation - RBF networks

Plan

1. Kernel density estimation
2. RBF networks

Reminder : Approaches to density estimation

- Parametric : assume that the data distribution follows a specific functional form governed by a small number of parameters.
- Non parametric : no assumption concerning the data distribution; construct local models centered on the data (e.g. kNN, KDE)
- Semi-parametric : construct local models on groups of data (RBF)

Reminder : the non parametric approach

- Locality idea: to estimate the probability of a new data point, we estimate the density in its neighborhood (e.g. histograms: bin; kNN : the hypersphere defined by the K nearest neighbors)
- The extend or the volume of the neighborhood is determined by the smoothness parameters (ex. histo: Δ_i ou Δ; KNN: K)
- To determine the neighborhood we can either:
  - fix the number of points K populating the region $\rightarrow$ KNN
  - fix the extend of the neighborhood $h \rightarrow$ kernel method (KDE = Kernel Density Estimation) = Parzen windows

Density estimation : basic idea, I

- In Probability Density estimation of an unknown density function $p(\mathbf{x})$ we want to:
  - estimate the probability $P(\mathbf{x} \in R)$ of a given region R of the data space
  - from a limited number, n, of training samples $\{\mathbf{x}_i | i = 1..n\}$
- If we knew the density function $p(\mathbf{x})$ then the probability would be given by:

Density estimation : basic idea, I

- In Probability Density estimation of an unknown density function $p(\mathbf{x})$ we want to:
  - estimate the probability $P(\mathbf{x} \in R)$ of a given region R of the data space
  - from a limited number, n, of training samples $\{\mathbf{x}_i | i = 1..n\}$
- If we knew the density function $p(\mathbf{x})$ then the probability would be given by:

$$P(\mathbf{x} \in R) = \int_R p(\mathbf{x}) d\mathbf{x}$$

Density estimation : basic idea, II

- The probability that for n draws we get k instances $\in R$ is given by the binomial distribution

$$\binom{n}{k} P(\mathbf{x} \in R)^k (1 - P(\mathbf{x} \in R))^{n-k}$$

and the expected value of k is:

Density estimation : basic idea, II

- The probability that for n draws we get k instances $\in R$ is given by the binomial distribution

$$\binom{n}{k} P(\mathbf{x} \in R)^k (1 - P(\mathbf{x} \in R))^{n-k}$$

and the expected value of k is:

$$E[k] = nP(\mathbf{x} \in R) \tag{1}$$

Density estimation : basic idea, III

- If R is small enough then $p(\mathbf{x}) \sim$ constant and:

$$P(\mathbf{x} \in R) = \int_R p(\mathbf{x}) d\mathbf{x} \simeq p(\mathbf{x}) V_R \tag{2}$$

where $V_R = \int_R d\mathbf{x}$ is the size of the R region.

- Then from eqs 1, 2

$$\frac{E[k]}{n} = P(\mathbf{x} \in R) \simeq p(\mathbf{x}) V_R \Rightarrow p(\mathbf{x}) \simeq \frac{E[k]}{n V_R} \tag{3}$$

Density estimation : basic idea, IV

- The density is given by:

$$p(\mathbf{x}) \simeq \frac{E[k]}{nV_R}$$

- $E[k]$ and V_R are unknown. Two approaches to get $p(\mathbf{x})$:
  - fix the volume, V_R, and get an estimate of $E[k]$, *kernel-based* density estimation.
  - fix $E[k]$ and compute the corresponding volume, k-*nn* density estimation

see (Duda, Hart, Stork, 2001, chap. 4.2) and (Bishop, 2006, chap 2.5)

Kernel density estimation

- In kernel density estimation the estimation of $p(\mathbf{x})$ is done *locally* for each new example $\mathbf{x}$ in its neighborhood R using the training set.
- In its simplest form it can be thought as a simple counting of the instances that fall in the neighborhood of $\mathbf{x}$

Kernel density estimation, a simple example, I

- A simple example:
- assume that R is a hypercube with edge length h then

$$V_R$$

Kernel density estimation, a simple example, I

- A simple example:
- assume that R is a hypercube with edge length h then

$$V_R = h^d$$

where d is the dimensionality of our space.

- Lets define a *kernel function* $K(\mathbf{x})$ as follows:

$$\begin{aligned} K(\mathbf{x}) &= 1 \text{ if } |x_j| \leq 1/2, \forall j = 1, \ldots, d \\ K(\mathbf{x}) &= 0 \text{ otherwise} \end{aligned} \tag{4}$$

what does it do?

Kernel density estimation, a simple example, I

- A simple example:
- assume that R is a hypercube with edge length h then

$$V_R = h^d$$

where d is the dimensionality of our space.

- Lets define a *kernel function* $K(\mathbf{x})$ as follows:

$$\begin{aligned} K(\mathbf{x}) &= 1 \text{ if } |x_j| \leq 1/2, \forall j = 1, \ldots, d \\ K(\mathbf{x}) &= 0 \text{ otherwise} \end{aligned} \tag{4}$$

what does it do?

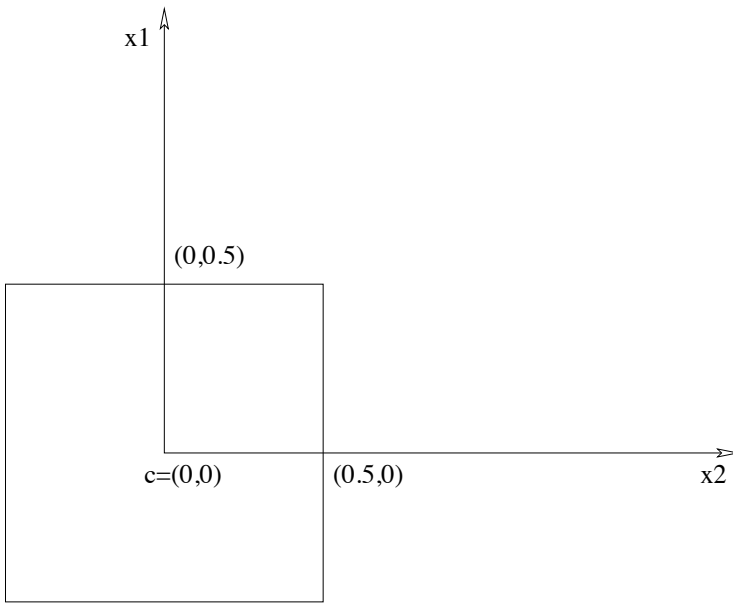


Figure: Hypercube centered at 0 with edge 1, defined by $K(\mathbf{x})$ eq 4

Kernel density estimation, a simple example, II

- Given $K(\mathbf{x})$ defined in eq 4 then

$$K(\frac{\mathbf{x}-\mathbf{x}_i}{h}) \tag{5}$$

  is equal to one if $\mathbf{x}_i$ falls within the R hypercube of volume $V_R = h^d$ centered at $\mathbf{x}$ and it is zero otherwise.
- So the expected number of training points $\mathbf{x}_i$ within the neighborhood R of $\mathbf{x}$ is given by:

$$E[k] =$$

Kernel density estimation, a simple example, II

- Given $K(\mathbf{x})$ defined in eq 4 then

$$K(\frac{\mathbf{x}-\mathbf{x}_i}{h}) \tag{5}$$

  is equal to one if $\mathbf{x}_i$ falls within the R hypercube of volume $V_R = h^d$ centered at $\mathbf{x}$ and it is zero otherwise.
- So the expected number of training points $\mathbf{x}_i$ within the neighborhood R of $\mathbf{x}$ is given by:

$$E[k] = \sum_{i=1}^{n} K(\frac{\mathbf{x}-\mathbf{x}_i}{h}) \tag{6}$$

- and from eq 3 we have:

$$p(\mathbf{x}) \simeq \frac{1}{nh^d} \sum_{i}^{n} K(\frac{\mathbf{x}-\mathbf{x}_i}{h}) \tag{7}$$

Single variable kernel density estimation, the "algorithm"

Given $TRN = \{x_i\}$ et x and a new instance x. To estimate $p(x)$:

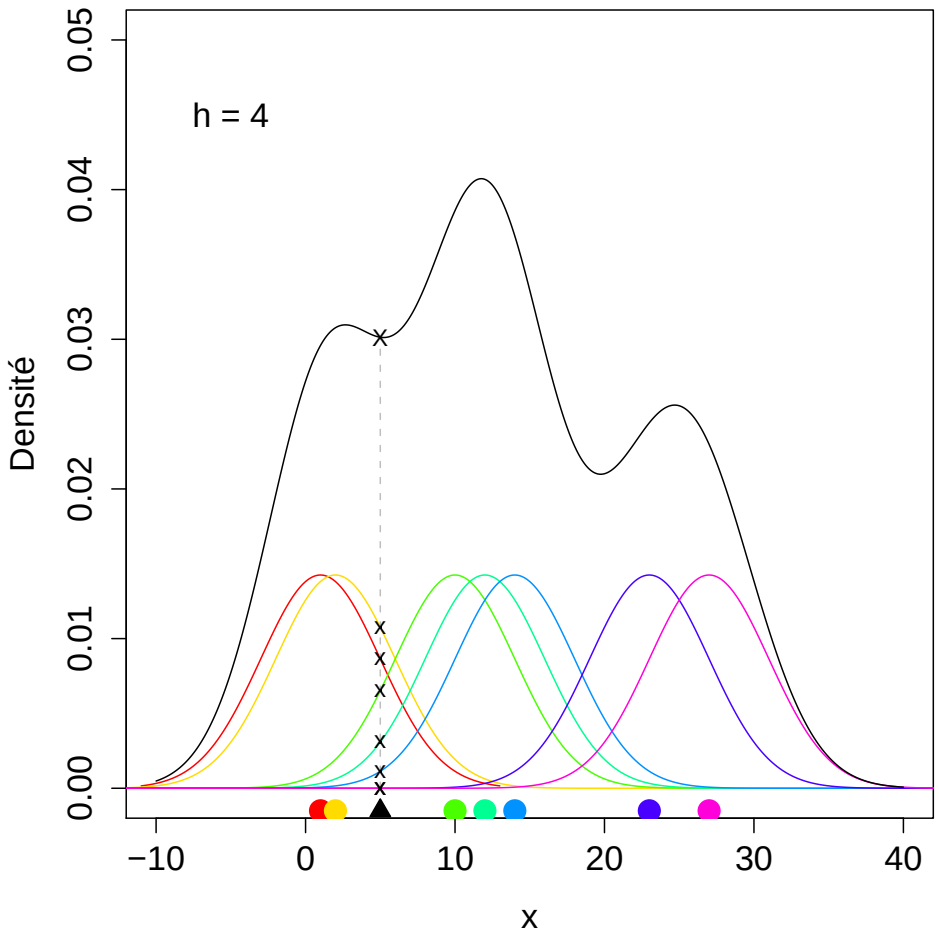
- We center the kernel K on each x_i.
- Each x_i constributes to the $p(x)$ density according to its distance from x
- The estimated density at point x $\propto$ to the sum of the contributions of all x_i

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

(note that x here is scalar)

<u>Comment</u>: The kernel can also be written as $K(x, x_i)$ to allow more freedom in the computation of distance/similarity between x and x_i $\rightarrow$ larger family of learning methods based in kernels (e.g. SVMs)

Gaussian kernel centered on 7 univariate instances



TRN = {1,2,10,12,14,23,27}

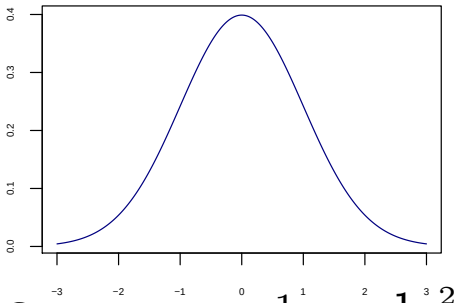
new $x = 5$

$\hat{p}(x) = 0.03$

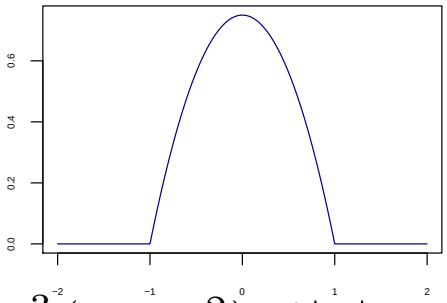
$=$ sum of the densities of the 7 kernels at point $x = 5$

Kernels for continuous variables

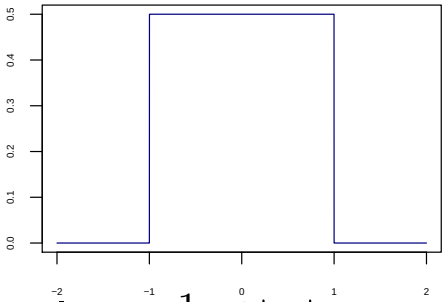
The kernel function $K(x)$ is in general symmetric and such that $\int_{-\infty}^{\infty} K(x)dx = 1$.



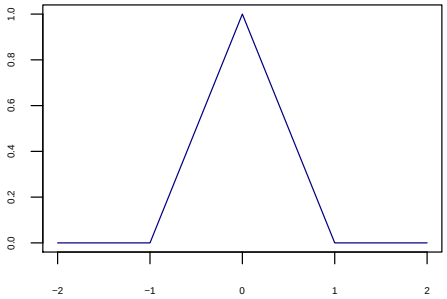
Gausian : $\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$



Epanech.: $\frac{3}{4}(1 - x^2)$ if $|x| < 1$, 0 if not



Rectangular : $\frac{1}{2}$ if $|x| < 1$, 0 if not



Triangulaire : $1 - |x|$ if $|x| < 1$, 0 if not

Kernels for discrete variables

- Binary variable

$$K(x, x_i) = \begin{cases} \lambda & x = x_i \\ 1 - \lambda & x \neq x_i \end{cases}$$

It should hold $0.5 < \lambda \leq 1$ so that we have a large K in case of equality and small otherwise.

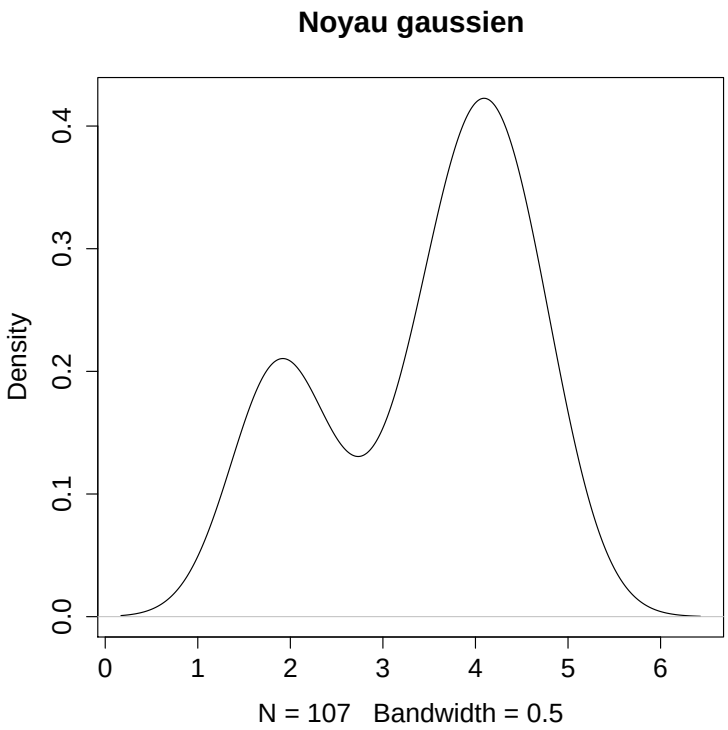
- Discrete variable with $g > 2$ values

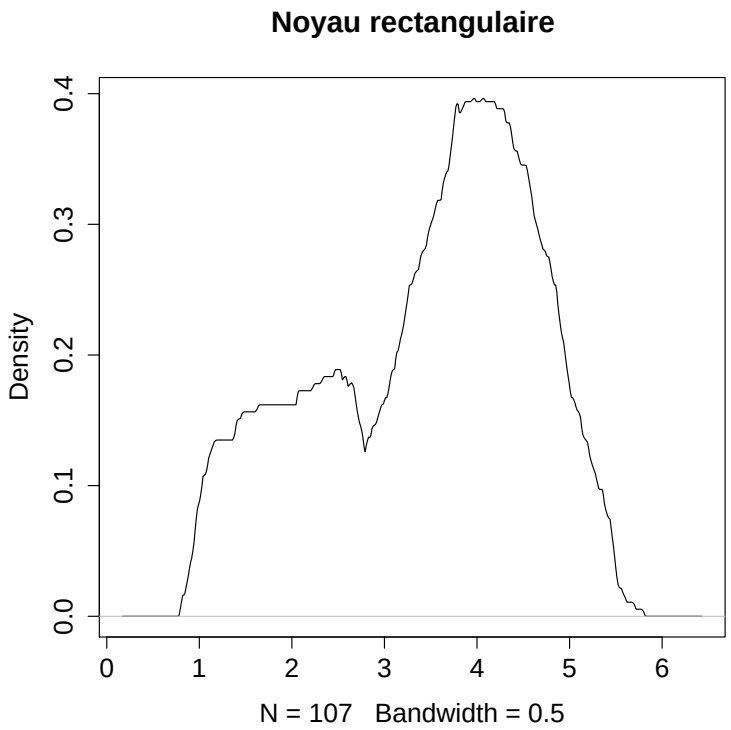
$$K(x, x_i) = \begin{cases} \lambda & x = x_i \\ \frac{1-\lambda}{g-1} & x \neq x_i \end{cases}$$

$g - 1 =$ normalization factor so that it holds $\sum_{j=1}^{g} K(.)$=1

Effect of the kernel form

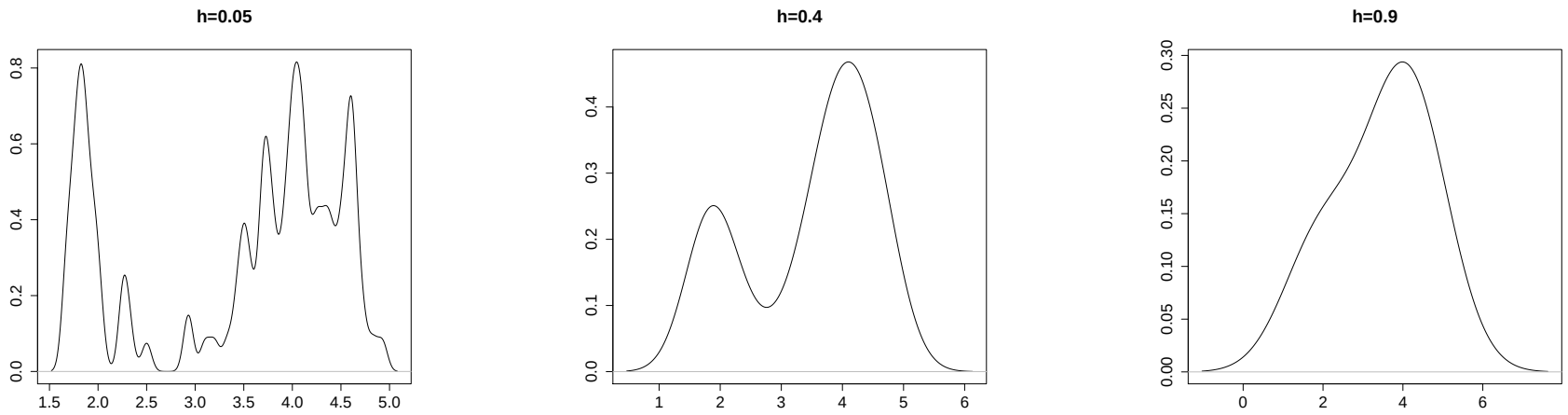
Data: duration in minutes of 107 eruptions of the Old Faithful Geyser, USA





Silverman, 1986

Effect of the kernel width



- h : controls the smoothness of the kernel, the larger the smoother.
- if h small : sensitive to noise; if h very big: we loose important characteristics (bimodality above)
- Kernel size is in general more important than its form.

k-NN based density estimation, I

- In kernel-based density estimation the volume is fixed over all kernels/instances
- However:
  - In high data density regions a large h over-smooths and might loose structure
  - reducing h would lead to noisy estimates in low density regions
  - i.e. optimal size depends on data density/data region
- k-NN based density estimation fixes that:
  - data density is kept fixed, not the volume

k-NN based density estimation, I

- Remember $p(\mathbf{x}) = \frac{E[k]}{nV_R}$
- We are given: n training instances, with n_i the instances of the c_i class, $\sum_i n_i = n$.
- We fix k
- The volume is now the volume of the sphere with the k-NNs centered at $\mathbf{x}$
- Among the k-NNs k_i belong to class c_i then:

$$p(\mathbf{x}|c_i) =$$

k-NN based density estimation, I

- Remember $p(\mathbf{x}) = \frac{E[k]}{nV_R}$
- We are given: n training instances, with n_i the instances of the c_i class, $\sum_i n_i = n$.
- We fix k
- The volume is now the volume of the sphere with the k-NNs centered at $\mathbf{x}$
- Among the k-NNs k_i belong to class c_i then:

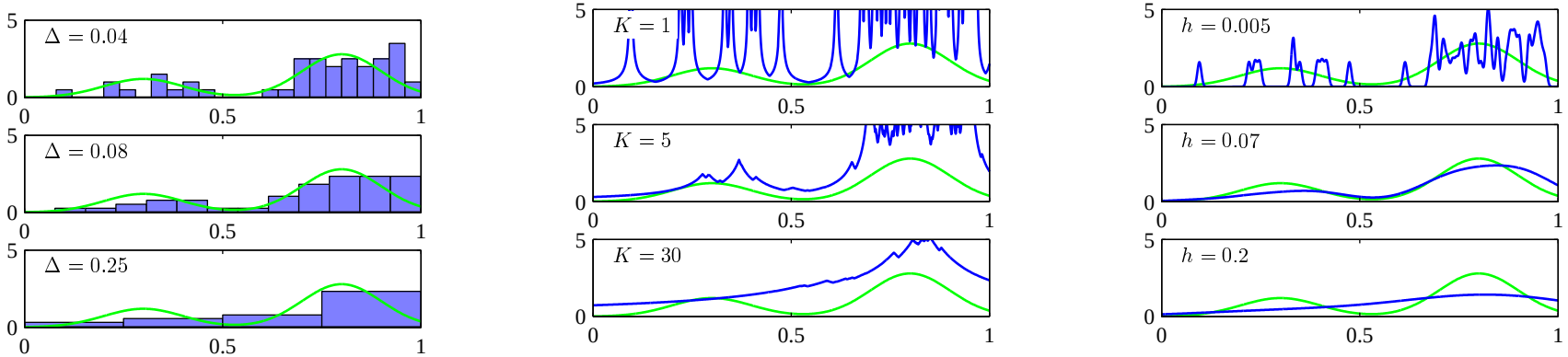
$$p(\mathbf{x}|c_i) = \frac{k_i}{n_i V_R},\ p(\mathbf{x}) = \frac{k}{nV_R},\ p(c_i) = \frac{n_i}{n} \Rightarrow$$

$$p(c_i|\mathbf{x}) = \frac{p(\mathbf{x}|c_i)p(c_i)}{p(\mathbf{x})} = \frac{k_i}{k}$$

which we already saw.

Impact of smoothness parameter on the $p(\mathbf{x})$ estimate

Histograms **k-NN** **KDE**



The smoothness parameter has a major impact on the results: it should not be very large nor very small $\rightarrow$ model selection problem.

Multivariate density estimation

- General solution: product of univariate kernels

$$\hat{p}(\mathbf{x}) = \frac{1}{nh_1 \dots h_d} \sum_{i=1}^{n} \prod_{j=1}^{d} K_j \left(\frac{x_j - x_{ij}}{h_j} \right)$$

- We can simplify using $K_j = K$, and $h_j = h$ for all dimensions:

$$\hat{p}(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^{n} \prod_{j=1}^{d} K \left(\frac{x_j - x_{ij}}{h} \right)$$

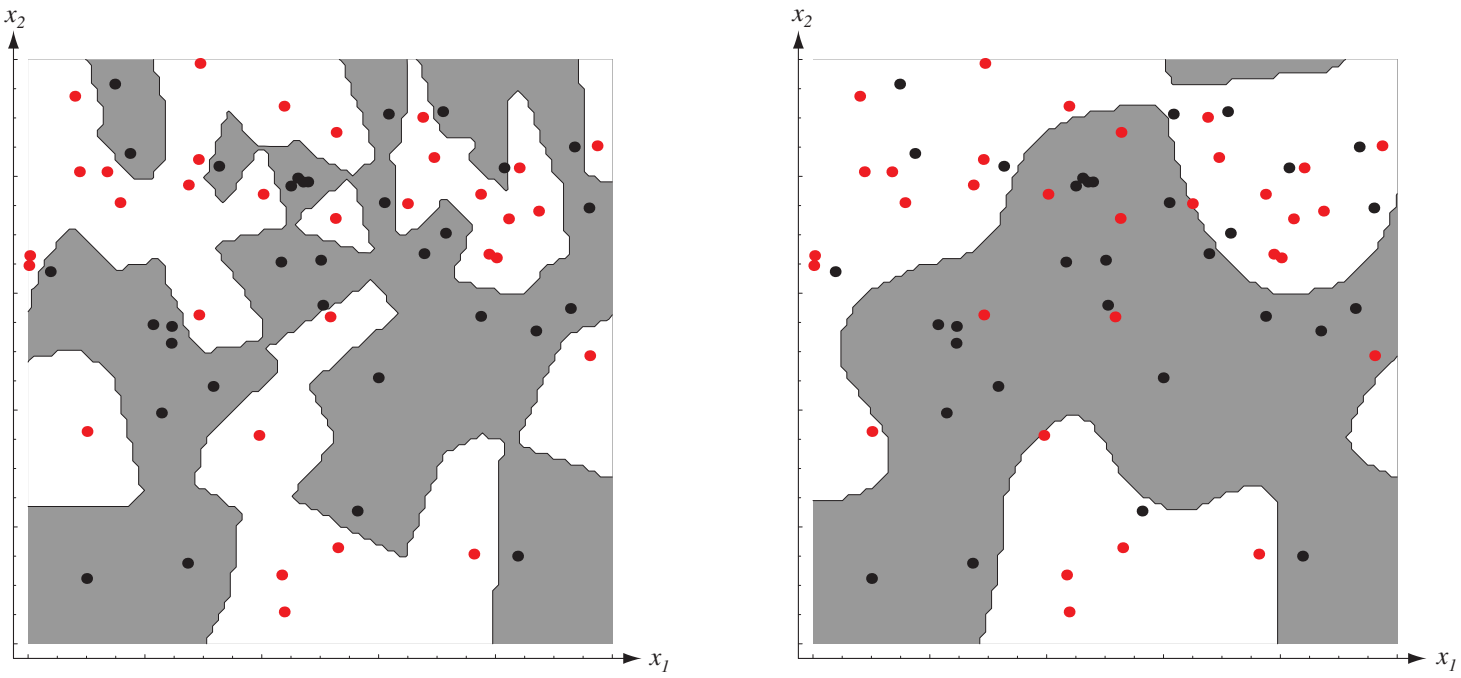
Classification using kernels

- Data : $TRN = \{\mathbf{x}_i\}$ pour $i = 1..n$, $\mathbf{x} =$ instance to classify
- Parameters : function K, size h
- General Algorithm:

Classification using kernels

- Data : $TRN = \{\mathbf{x}_i\}$ pour $i = 1..n$, $\mathbf{x}$ = instance to classify
- Parameters : function K, size h
- General Algorithm:
  1. For each class $c \in \mathcal{C}$
  2. $\quad P(c) \leftarrow \frac{n_c}{n}$
  3. $\quad TRN_c = \{\mathbf{x}_i | \mathbf{x}_i \in c\}$
  4. $\quad \hat{p}(\mathbf{x}|c) = \frac{1}{n_c h_1 \ldots h_d} \sum_{i=1}^{n_c} \prod_{j=1}^{d} K_j \left(\frac{x_j - x_{ij}}{h_j} \right)$
  5. $\quad P(c|\mathbf{x}) \leftarrow \hat{p}(\mathbf{x}|c) P(c)$
  6. Return c having maximal $P(c|\mathbf{x})$
- Actually Naive Bayes: where we estimate $p(\mathbf{x}|c)$ for each class c, and then using Bayes theorem to calculate $p(c|\mathbf{x})$.

Effect of kernel width on the decision surfaces



- Small h (left) $\rightarrow$ complex decision surfaces
- Large h (right) $\rightarrow$ smoother/simpler decision surfaces.

KNN vs Kernels

- Both k-NN and kernel density estimates converge to the true probabilities as $n \to \infty$ provided that V_R shrinks with n and k grows with n.

- influence zone determined by the number of neighbors K
- shape of neighborhood determined by the distance function
- $P(c|\mathbf{x})$ estimated directly using the proportion of the c class in the K neighbors.
- only the K nearest neighbors are used to estimate $P(c|\mathbf{x})$, even though all instances $\mathbf{x}_i$ are checked to identify the K neighbors.
- As $n \to \infty$ k-NN classification error is upper bounded by twice the Bayes error.

- influence zone determined by the size of h
- shape of neighborhood determined by the kernel function
- we first estimate $p(\mathbf{x}|c)$ and then use it to get $P(c|\mathbf{x})$
- all $\mathbf{x}_i$ are used to estimate the density at point $\mathbf{x}$ their contribution $\searrow$ with h

Non parametrique methods: summary

- Advantages
  - No assumption on the form of the data distribution
  - Guaranteed convergence if sufficient number of instances
- Limitations
  - Need to store all training instances
  - Slow classification time if large |TRN|
  - Sensitive to the choise of the hyperparameters k (k-NN) and h (kernel)
  - Curse of dimensionality: if many variables then we need many training instances for the methods to work

Plan

1 Kernel methods

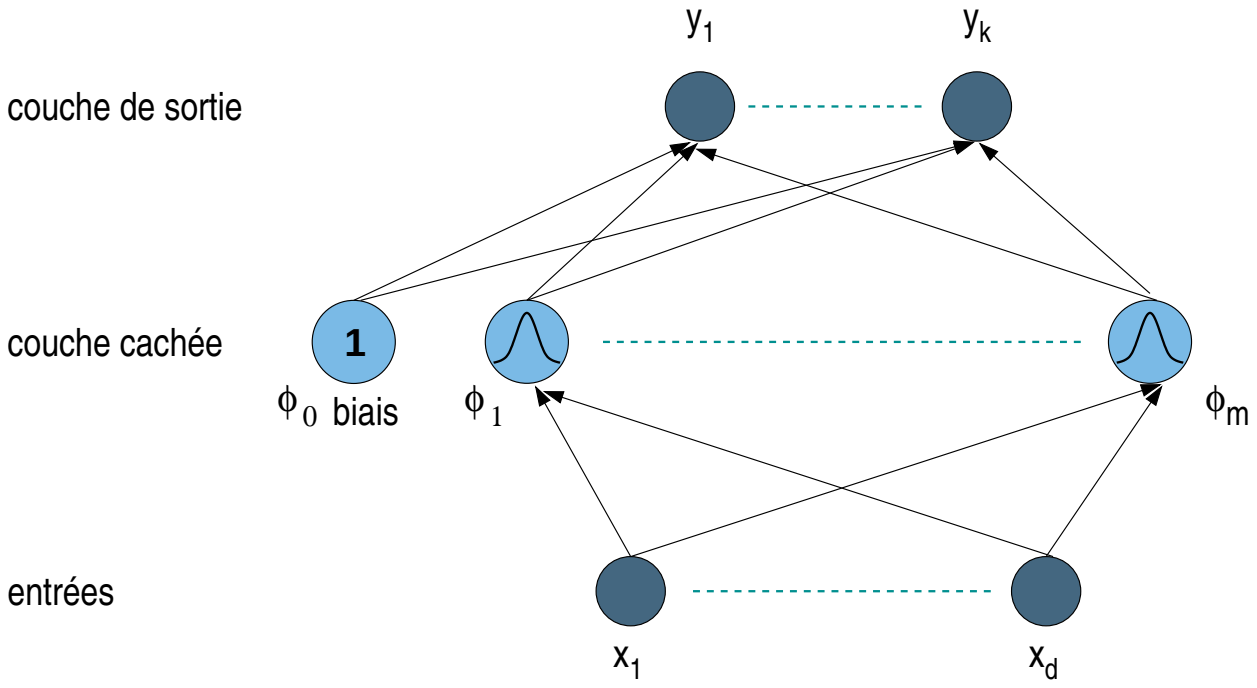
2 RBF networks

Semi parametric densite estimation

- Main problem of non-parametric methods: the size of the model grows linearly with the number of instances
- Semi-parametric approach: we assume a general class of functional forms.
  - more flexible than the parametric approach: allows for a mixture of models.
  - costs less than the non parametric approach: # of models $\ll$ # of examples
  - Radial Basis Function networks: use m basis functions, $m \ll n$

RBF network structure

An RBF net generates a global approximation of the target function through a linear combination of the m local basis functions.



The hidden layer

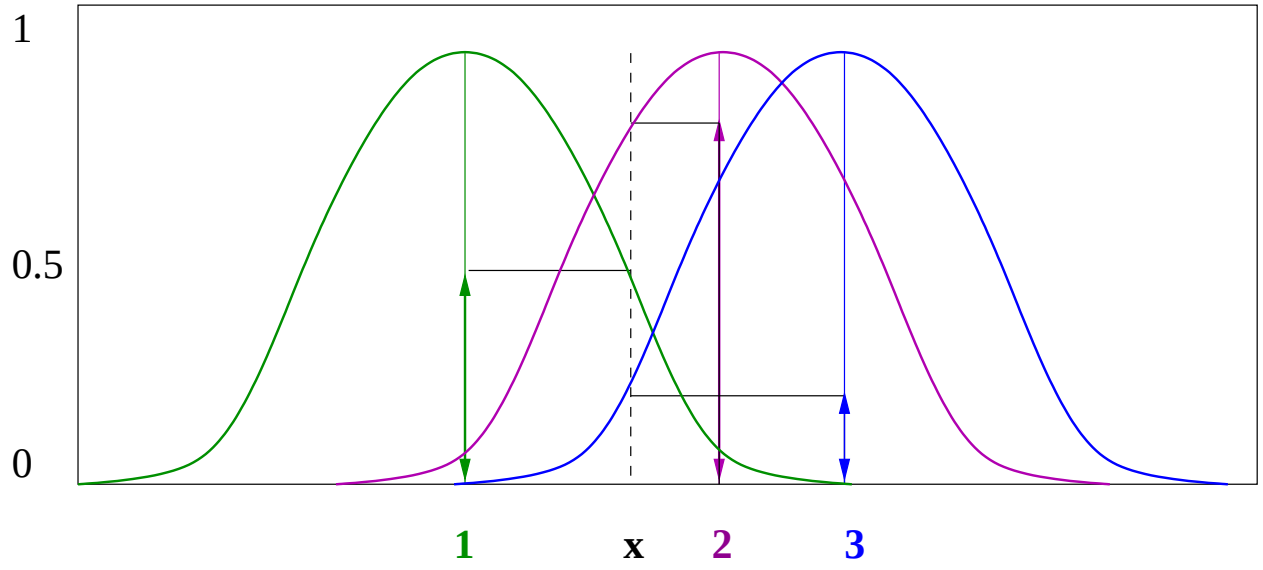
- contains m hidden units, each one having a prototype vector $\boldsymbol{\mu}$ of the same dimensionality as the input vectors $\mathbf{x}$.
- $\phi_j(\mathbf{x})$: activation of a hidden unit = function of the distance between the $\mathbf{x}$ and the $\boldsymbol{\mu}$
- the most often used $\phi_j(\mathbf{x})$ is the gaussian

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_j\|^2}{2\sigma_j^2}\right)$$

- More general, σ_j can be replaced by a covariance matrix

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1}(\mathbf{x} - \boldsymbol{\mu}_j)\right)$$

Hidden units activation



- 3 hidden units of the same width with centers 1, 2, 3 respectively
- input to classify $\mathbf{x}$: dotted line
- as $\mathbf{x}$ gets closer to the center of a gaussian its activation gets larger (height of arrows)

Output layer

- Input : activations of the m hidden units (more or less large as a function of the distance of the input vector from the center of each unit.
- Output : linear combination of the activations of the hidden layer

$$\begin{aligned} y_k(\mathbf{x}) &= \sum_{j=1}^{m} w_{kj}\phi_j(\mathbf{x}) + \beta_k \\ &= \mathbf{w}_k^T \mathbf{\Phi}(\mathbf{x}) + \phi_{k0} \end{aligned}$$

$$\mathbf{\Phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \ldots, \phi_m(\mathbf{x}))^T$$

$y_k =$ the value of the output of the k unit for $k = 1 \ldots c$
$w_{kj} =$ connection weights going from the hidden unit j to the output k
$\phi_{k0} =$ bias term of the k output

We can include the bias term ϕ_{k0} in the weights by adding one more hidden unit $\phi_{k0} = 1$

$$y_k(\mathbf{x}) = \sum_{j=0}^{m} w_{kj}\phi_j(\mathbf{x}) = \mathbf{w}_k^T \mathbf{\Phi}(\mathbf{x})$$

Training of RBFs

Hybrid learning

- Phase 1
  - Target : hidden layer
  - Unsupervised learning
  - Parameters to learn : positions of the centers.
- Phase 2
  - Target : output layer
  - Supervised learning
  - Parameters to learn : coefficients of the linear model

Phase 1 : training the hidden layer

- hidden layer : unsupervised learning
- data : TRN (target values are not used in this phase)
- goal : distribute the centers $\boldsymbol{\mu}_j$ in the data space in such a manner as to
  - cover all regions proportionally to their density
  - have an adequate level of overlap
- hyperparametres
  - m : number of centers/prototypes/hidden units
  - σ_{min} : minimum size of the radius σ_j of each hidden unit

Stages of phase 1

1. Determine the coordinates of the m centers $\boldsymbol{\mu}_j$.
   - sample randomly m instances from TRN and use them as prototypes
   - use a clustering method such as K-means to find the prototypes
2. Use the m prototypes $\boldsymbol{\mu}_j$ to calculate the activation of each hidden unit

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\left\|\mathbf{x} - \boldsymbol{\mu}_j\right\|^2}{2\sigma_j^2}\right)$$

The σ_j are adjusted so the the basis functions overlap in order to produce a smooth function of the data.
Alternatively we can have a full covariance matrix $\boldsymbol{\Sigma}_j$ per center.

Phase 2 : training the output layer

- At the end of phase 1 :
  the hidden layer performed a non-linear transformation of the inputs to a new space of dimensionality m
- The role of the output layer:
  create a linear model that approximates the target function in the new space
  - inputs : the m activations of the hidden units
  - outputs :
    - regression : a neuron representing the value to estimate
    - classification : c output neurons for c binary classes

Optimisation of the linear model

- goal of phase 2 : adjust the coefficients of the linear model (=weights from the hidden units to the output units) to minimise the sum of squares, can do it:
  - either through the closed form solution of linear regression
  - or by gradient descent
- in each case the coefficients/weights are obtained by minimizing the quadratic error (t_{ik} target value)

$$E = \frac{1}{2} \sum_{i=1}^{n} \sum_{k=1}^{c} (y_k(\mathbf{x}_i) - t_{ik})^2$$

Model complexity

- Phase 1 : the number of parameters to fit depends on the way we choose
  - σ_j for each center $\rightarrow m(d+1)$
  - $\mathbf{\Sigma}_j$ covariance matrix for each center $\rightarrow md(d+3)/2)$
- Phase 2 : $m * k$ coefficients of the linear combinations

RBF nets : summary

- Given a target function to approximate the RBF nets:
  - construct m local and redundnant approximations, and then
  - they combine them linearly in order to obtain a global relatively smooth approximation
- In general they learn faster than the multi-layer perceptrons because the two layers are trained separately.
- like all distance based methods
  - they are quite sensitive to non relevant/correlated variables
  - in general they do not work well in high dimensionalities
- Same model structure as in Support Vector Machines
- Kind of obselete model

References

- C. Bishop (2006). Pattern Classification and Machine Learning, Section 2.5.
- C. Bishop (1995). Neural Networks and Pattern Classification, Chapitres 2 et 4.
- R. Duda, P. Hart et D. Stork (2001). Pattern Classification, Chap. 4.
- B. Silverman (1986). Density Estimation. Chap. 1.